



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és Adatszerkezetek I.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2026. március 29.

Tartalom I

- 1 Bináris fa
 - Ábrázolása
 - Bejárási módszerek
 - Magassága
 - Levelek száma
- 2 Bináris fa (feladatok)
 - Maximum kulcs
 - Negatív kulcs
- 3 Szintfolytonos bejárás
 - Szintek
 - Gyerekek száma
 - Negatív kulcs (szintfolytonosan)
 - Levelek száma (szintfolytonosan)
- 4 Kifejezés fa

Bináris fa

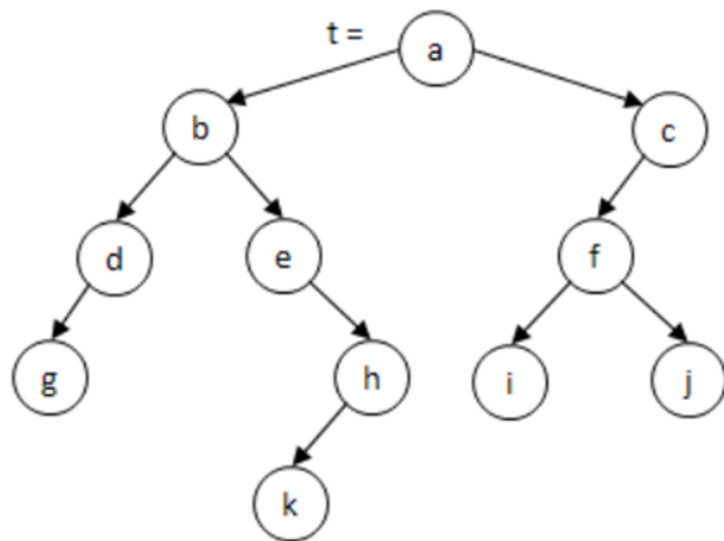
Bináris fák ábrázolása

• Láncolt ábrázolás

- Két pointerrel: `bal(left)`, `jobb(right)`
- Három pointerrel: `bal(left)`, `jobb(right)`, `szülő(parent)`

• Aritmetikai ábrázolás

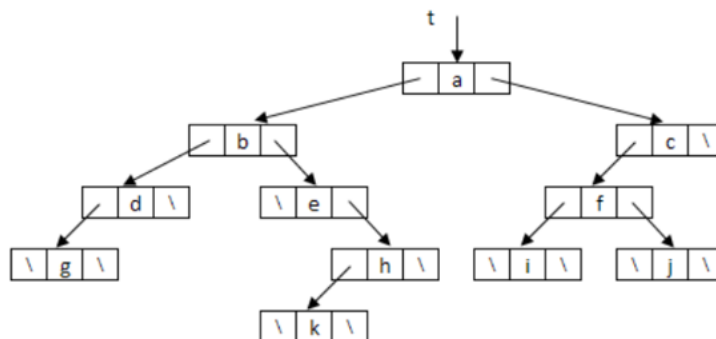
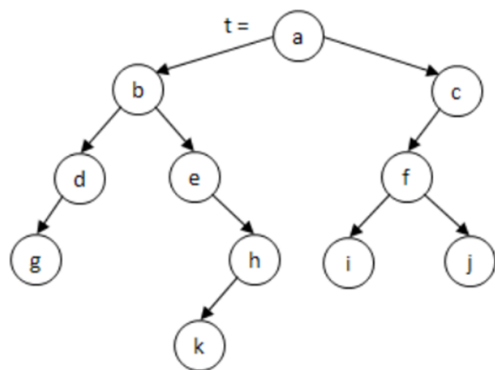
- Szintfolytonos elhelyezés egy tömbben.
- Bal-, jobb gyerek, szülő az indexekből számítható ki.
- Csak „tömör” fákra hatékony, lásd kupac.
- Kupacoknál fogjuk tárgyalni.



Két pointerrel történő ábrázolás

Elem típus:

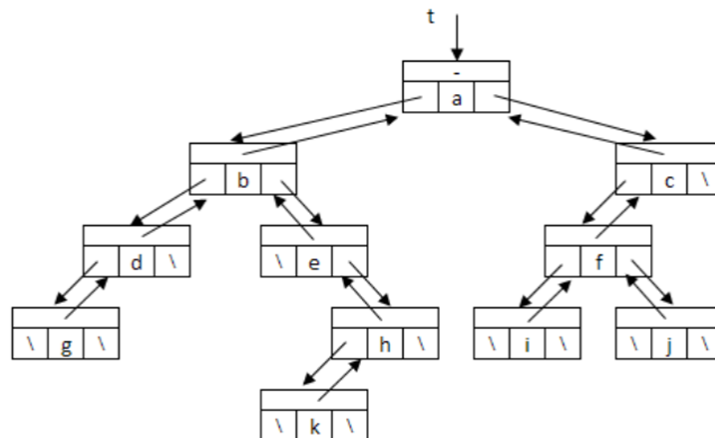
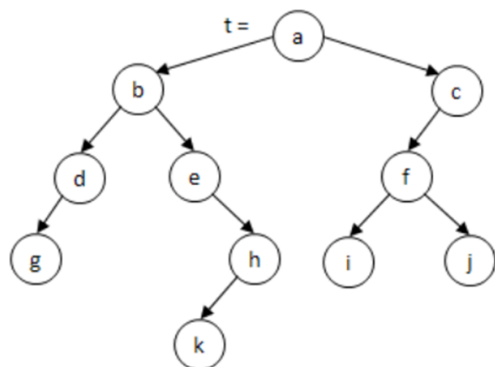
Node
+ $key : \mathcal{T}$ // $\mathcal{T}$ valamilyen ismert típus
+ $left, right : Node^*$
+ $Node() \{ left := right := \emptyset \}$ // egycsúcsú fát képez belőle
+ $Node(x : \mathcal{T}) \{ left := right := \emptyset ; key := x \}$



Három pointerrel történő ábrázolás

Elem típus:

Node3
+ $key : \mathcal{T}$ // $\mathcal{T}$ valamilyen ismert típus
+ $left, right, parent : Node3^*$
+ $Node3(p:Node3^*) \{ left := right := \odot ; parent := p \}$
+ $Node3(x : \mathcal{T}, p:Node3^*) \{ left := right := \odot ; parent := p ; key := x \}$



Algoritmusokban használt jelölések

Ábrázolás szintjén (megadott művelet)	Adatszerkezet szintjén (megadott művelet)	Jelentése
$t \rightarrow left$	$left(t)$	t bal oldali részfája
$t \rightarrow right$	$right(t)$	t jobb oldali részfája
$t \rightarrow parent$	$parent(t)$	t szülője
$t \rightarrow key$	$key(t)$	t csúcs kulcsa
$t = 0, NIL, NULL$	$t = \Omega$	t üres fa

Bináris fák bejárása

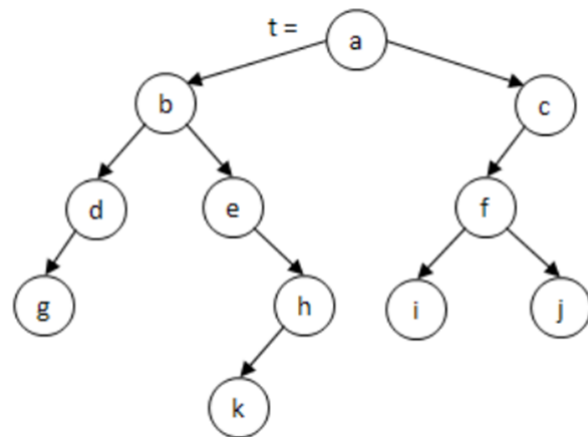
- Bejárás a csúcsok felsorolását jelenti, tehát a programozásból tanult felsorolónak tekinthető. Mint azt később látni fogjuk, bármelyik felsorolós programozási tétel működik ezek segítségével.

- **Rekurzív bejárások**

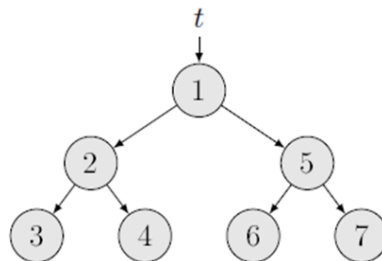
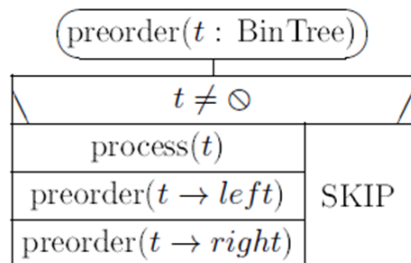
- Preorder
- Inorder
- Postorder

- **Szintfolytonos bejárás:**

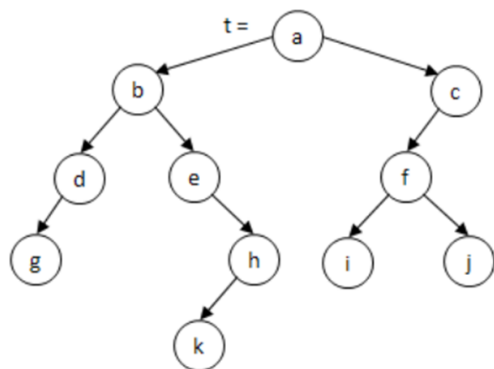
- Általános gráfokon használatos szélességi bejárás rokona.



Preorder bejárás



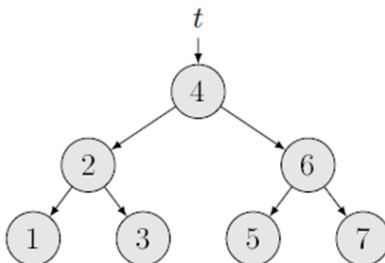
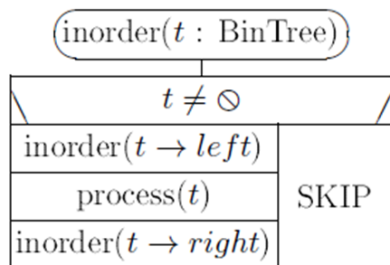
Elemek elérésének sorrendje.



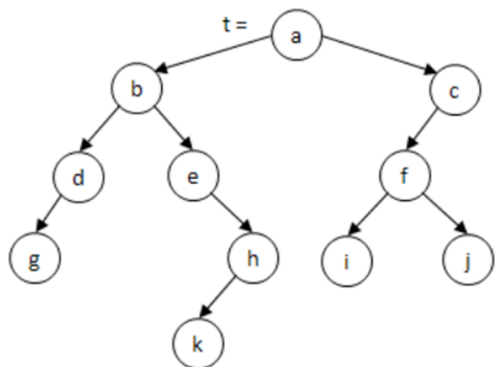
Preorder bejárással kapott sorrend:

a b d g e h k c f i j

Inorder bejárás



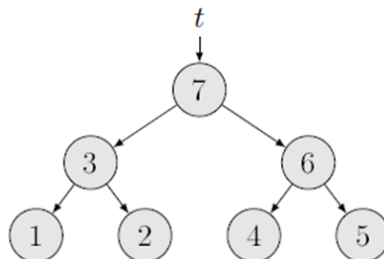
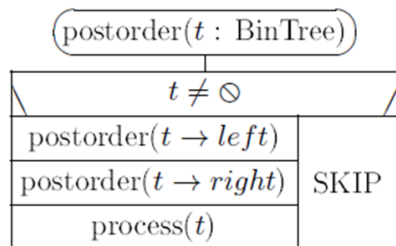
Elemek elérésének sorrendje.



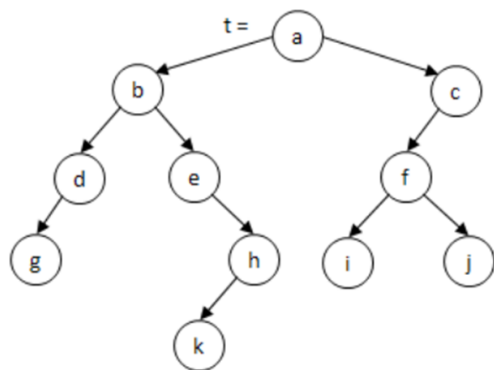
Inorder bejárással kapott sorrend:

g d b e k h a i f j c

Postorder bejárás



Elemek elérésének sorrendje.

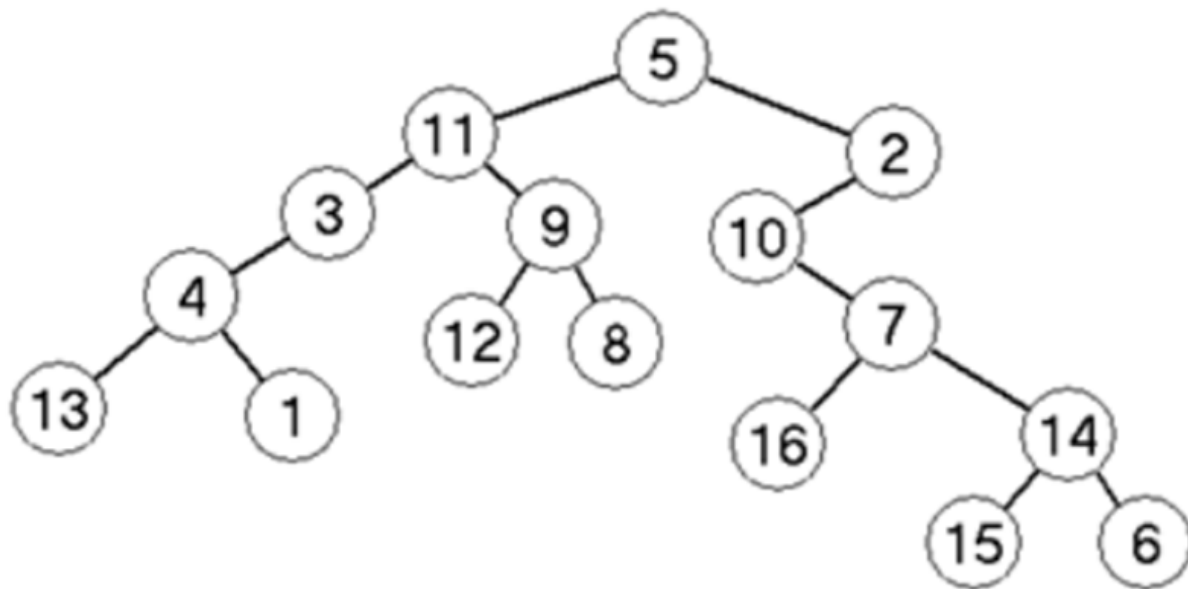


Postorder bejárással kapott sorrend:

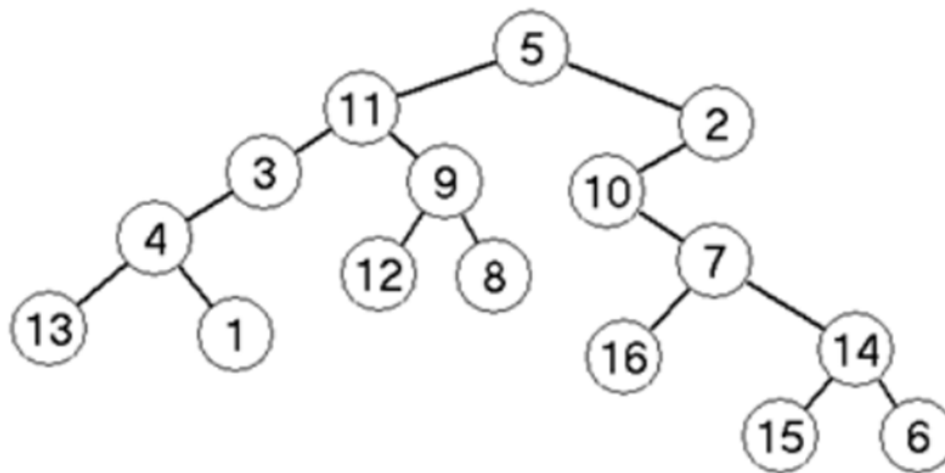
g d k h e b i j f c a

Feladat: Bináris fák bejárása 1

- Add meg milyen sorrendben írja ki az alábbi fa kulcsait a három rekurzív bejáró algoritmus



Feladat: Bináris fák bejárása 1



Preorder: 5, 11, 3, 4, 13, 1, 9, 12, 8, 2, 10, 7, 16, 14, 15, 6

Inorder: 13, 4, 1, 3, 11, 12, 9, 8, 5, 10, 16, 7, 15, 14, 6, 2

Postorder: 13, 1, 4, 3, 12, 8, 9, 11, 16, 15, 6, 14, 7, 10, 2, 5

Feladat: Bináris fák bejárása 2

- Rekonstruáld a fát az alábbi post- és inorder bejárásból!
 - POSTORDER: 17, 4, 7, 9, 45, 91, 2, 8, 53, 10, 66, 72, 34, 5, 3
 - INORDER: 17, 8, 4, 7, 91, 45, 9, 2, 3, 5, 34, 53, 66, 10, 72

Feladat: Bináris fák bejárása 2

- POSTORDER: 17, 4, 7, 9, 45, 91, 2, 8, 53, 10, 66, 72, 34, 5, 3
- INORDER: 17, 8, 4, 7, 91, 45, 9, 2, 3, 5, 34, 53, 66, 10, 72

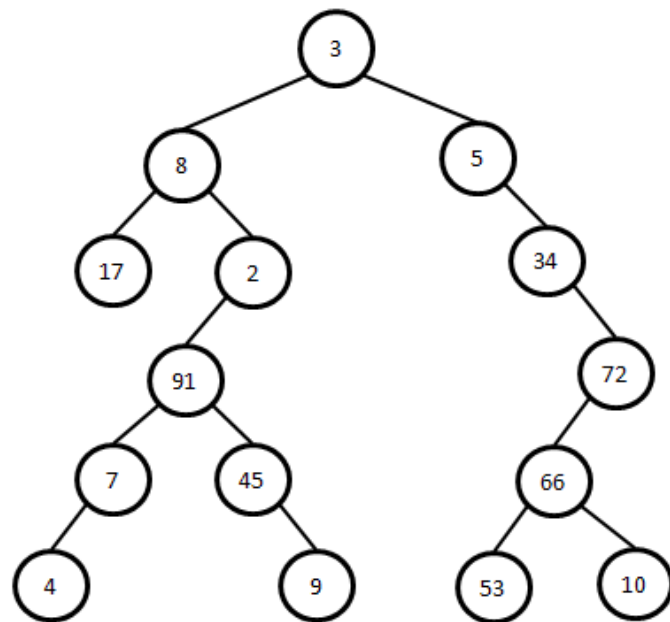
- Hogyan oldható meg?

- Fejtsük meg mi a fa gyökere?
- Melyik bejárás adja meg biztosan?
- **Postorder** (utolsónak írja ki):
3

- Mit tudhatunk meg az **inorder** bejárásból?

- 3-tól balra kiírt elemek a 3-as bal részfájában lesznek,
- a jobbra található elemek a jobb részfájában.

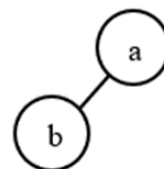
- A megoldás részletes bemutatása:



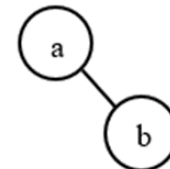
<https://people.inf.elte.hu/veanna/algo1/feladatgyujtemeny/binfa/abr05/megoldas.html>

Bármely kettő bejárásból rekonstruálható a fa alakja?

- **Nem!** Melyik kettőből nem lehet?
- **Válasz:** preorder és postorder pár nem elegendő!
- **Miért?**
 - Mert az egy gyerekes csúcsok esetén nem lehet eldönteni, hogy a gyerek bal, vagy jobb gyereke a csúcsnak!



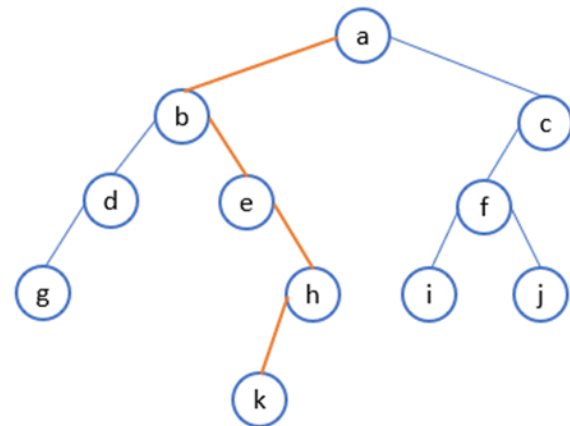
PRE: a b
POST: b a



a b
b a

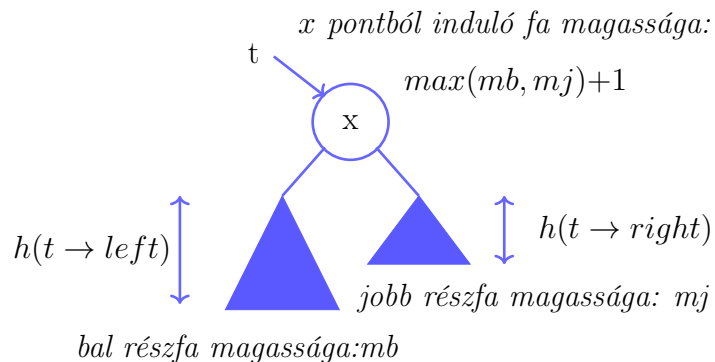
Bináris fa magassága

- Mit értünk **magasság** alatt:
A fa egyik leghosszabb ágának a hossza, azaz az ág éleinek száma.
- Üres fa magasságát -1-nek értelmezzük.
- Egy pontból álló fa magassága: 0
- Két pontból álló fa magassága: 1
- Példában szereplő fa magassága: 4

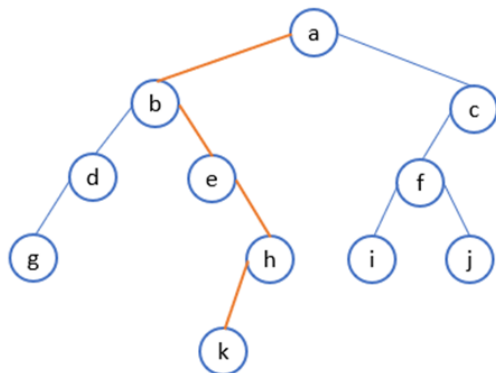


Bináris fa magassága

- Hogyan oldhatjuk meg?
- Ötlet: tetszőleges x pont magassága a következőképpen oldható meg rekurzívan:
- Határozzuk meg x bal részfájának magasságát, legyen ez mb .
- Határozzuk meg jobb részfájának a magasságát, legyen ez mj .
- Ebből x magassága: $\max(mb, mj) + 1$
- Üres fa magassága: -1
- Kész a megoldás!



Bináris fák magassága



$h(t:\text{Bintree}):Z$

$t \neq 0$	
$mb := h(t \rightarrow \text{left})$	return -1
$mj := h(t \rightarrow \text{right})$	
return $\max(mb, mj) + 1$	

Esetek ellenőrzése:

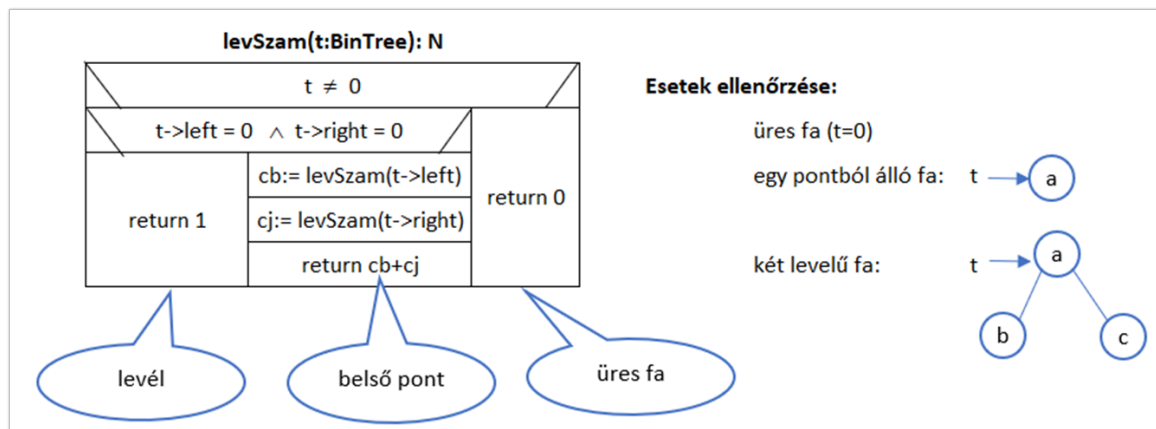
üres fa ($t=0$)

egy pontból álló fa: $t \rightarrow a$

két pontból álló fa: $t \rightarrow a \rightarrow b$

Bináris fa (feladatok)

Levelek számának meghatározása rekurzív algoritmussal



- Ellenőrizzük, hogy x levél-e:
 - ha igen, akkor visszatérünk 1-gyel
 - ha nem, akkor visszatérünk a bal és jobb részfájából kapott értékek összegével

Maximális kulcsának meghatározása 1

- **Első megoldás:** Rekurzív iterálással globálisan max-ot keresek.
- Bármelyik rekurzív bejárással, pl. *preorder* bejárást követve, egy max változót „viszünk magunkkal”, amelyet folyamatosan frissítünk, az aktuális elem kulcsának megfelelően. FONTOS, hogy max referencia szerint átadott paraméter!
- Mivel üres fára nem tudna megoldást adni, az üres fa esetét ki kell szűrni, hibát jelzünk ilyenkor.

maxKulcs(t:BinTree): T

t ≠ 0	
max:=t->key	Error
preorder_Max(t,max)	
return max	

preorder_Max(t:BinTree, &max:T)

t ≠ 0		skip
t -> key > max		
max := t->key	skip	
preorder_Max(t->left, max)		
preorder_Max(t->right, max)		

- Kívülről a **maxKulcs** eljárás hívható, **preorder_Max** csak egy segéd eljárás. A **maxKulcs** a fa gyökerében lévő kulccsal indítja el a bejárást.
- Ezzel a módszerrel bármelyik programozási tételt megvalósíthatjuk! A tétel eredmény változóit referencia szerinti paraméterek formájában visszük magunkkal. A másik két bejáró algoritmus is megfelelő lenne!

Maximális kulcsának meghatározása 2

- **Második megoldás:** Részfák részmegoldásait használom.
- Nem paraméterben, hanem visszatérési értéként állítjuk elő a maximumot.

maxKulcs(t:BinTree):T	
t=0	
Error	return(maxBejár(t))

maxBejár(t:BinTree):T				
t->left =0 és t->right=0				levél esete
return t->key	t->left≠0 és t->right≠0	t->left≠0 és t->right=0	t->left=0 és t->right≠0	belső pontok esetei
	max1:=maxBejár(t->left)	max1:=maxBejár(t->left)	max1:=maxBejár(t->right)	
	max2:=maxBejár(t->right)			
	max1:=max(max1,max2)			
return max(max1,t->key)				

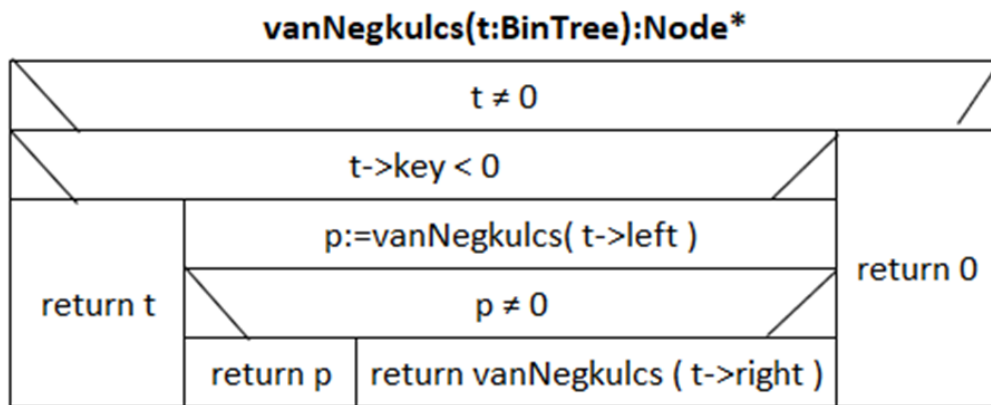
levél esete

belső pontok esetei

- Ilyenkor is szükség van egy „indító” eljárásra, mely az üres fa esetét kezeli.
- Ennek az algoritmusnak az az előfeltétele, hogy t nem üres fa. Ezért a rekurzív hívást csak a nem üres részfákra hajtjuk végre. Ezért kell a belső pontoknál a háromféle típust külön-külön kezelni.

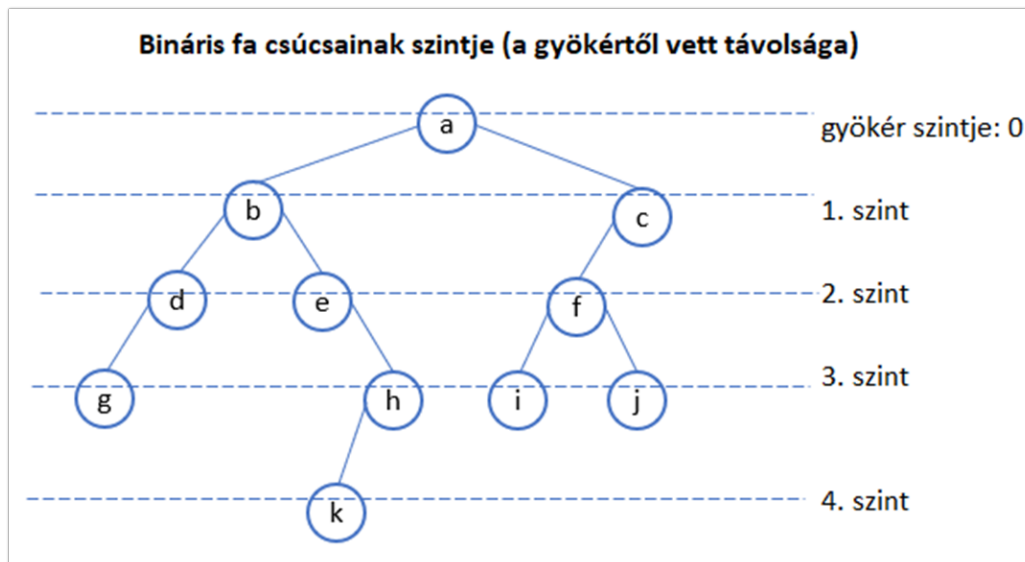
Negatív kulcs meghatározás rekurzívan

- **Feladatunk keresés.** Függvény alakban készítjük el. Bejárás közben, ha találkozunk egy negatív kulcsú elemmel, leállunk a bejárással, és visszatérünk az elem címével. Ha végigjártuk a fát, és nem talákoztunk negatív kulcsú elemmel, nullát adunk eredményül. Így nincs szükség a keresés logikai eredmény változójára.



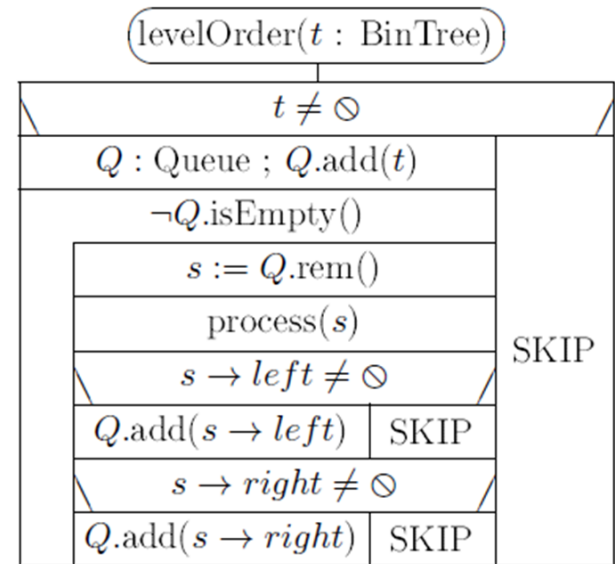
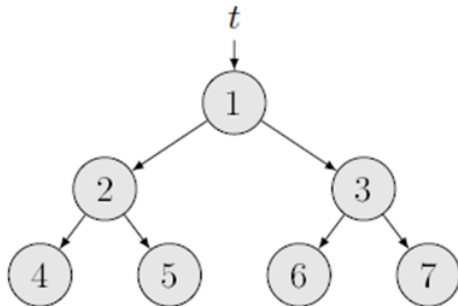
Szintfolytonos bejárás

Bináris fa szintjei, szintekkel kapcsolatos feladatok

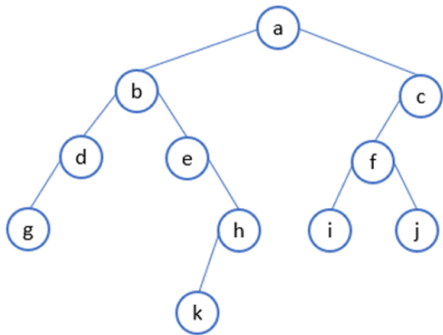


Láncoltan ábrázolt bináris fa szintfolytonos bejárása

- A jegyzetben található algoritmus
- A bejárás szintén felsorolóként használható, a fa mindegyik csúcsához eljut a feldolgozás
- Az elemek elérésének sorrendje:



Szintfolytonos bejárás szemléltetése

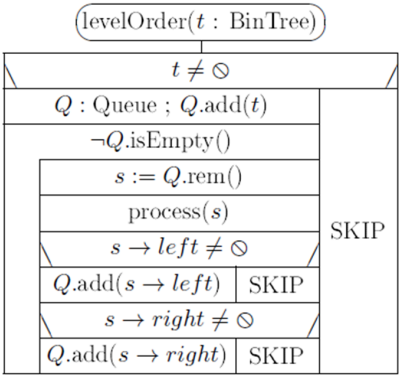


Ciklus indítása:

Q tartalma	
a	

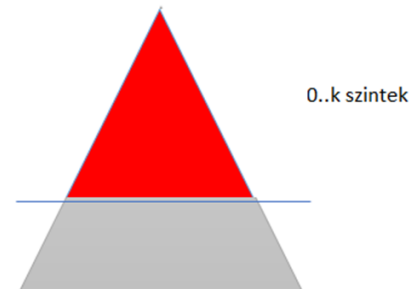
Feldolgozott csúcs

a	b c
b	c d e
c	d e f
d	e f g
e	f g h
f	g h i j
g	h i j
h	i j k
i,j,k	



Hány egy-gyerekes csúcsa van a 0..k szinteken?

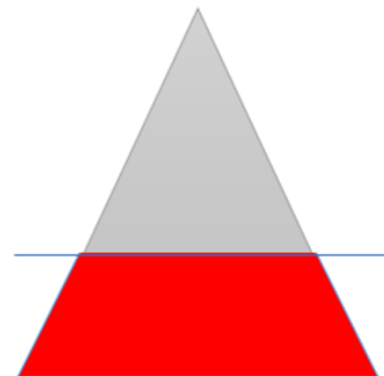
egyGyerek(t: Bintree, k: N) : N		
t = 0 vagy k < 0		
return 0	c1:= egyGyerek(t->left,k-1)	
	c2:= egyGyerek(t->right,k-1)	
	(t->left≠0 és t->right=0) vagy (t->left=0 és t->right≠0)	
	return c1+c2+1	return c1+c2



Hány egy-gyerekes csúcsa van a k-nál nagyobb szinteken?

egyGyerek(t:Bintree, k: N) : N

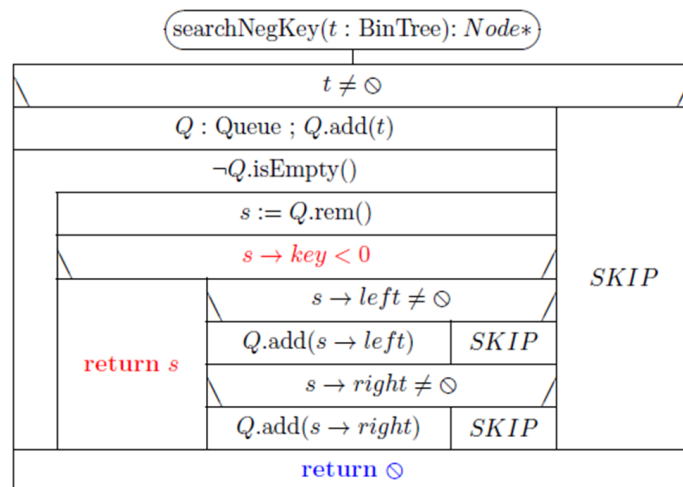
t = 0			
return 0	c1:= egyGyerek(t->left,k-1)		
	c2:=egyGyerek(t->right,k-1)		
	k < 0 és ((t->left≠0 és t->right=0) vagy (t->left=0 és t->right≠0))		
	return c1+c2+1		return c1+c2



k-nál nagyobb szintek

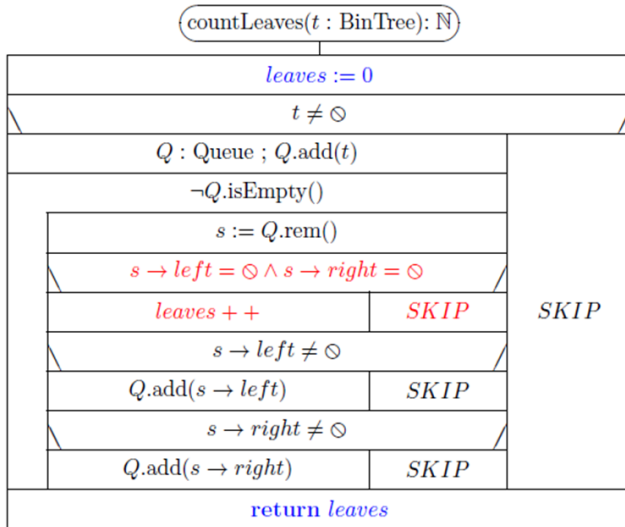
Negatív kulcs meghatározás szintfolytonos bejárással

- A fa kulcsai egész számok, keressünk negatív kulcsú elemet.
- A sikeres keresés eredménye az elemre mutató pointer.
- Sikertelen keresés esetén NULL pointert adunk vissza.

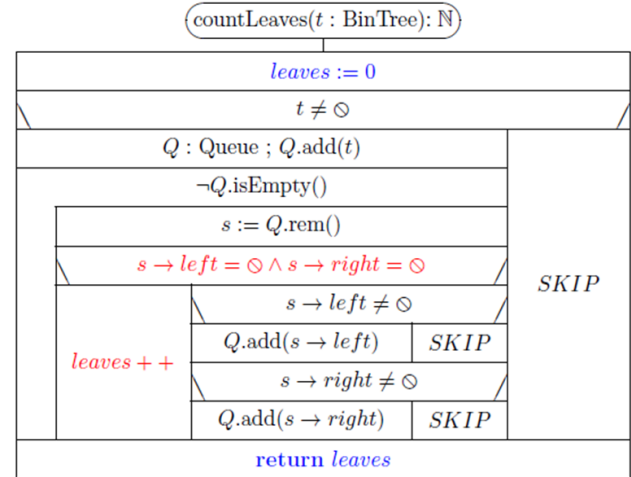


Levelek számának meghatározás szintfolytonos bejárással

- Számoljuk meg a fa leveleit:



$process(t)$ = ha t egy levél, növeljük a számlálót



kicsit ügyesebben is csinálhatjuk: ha t levél, úgysem lesznek leszármazottjai

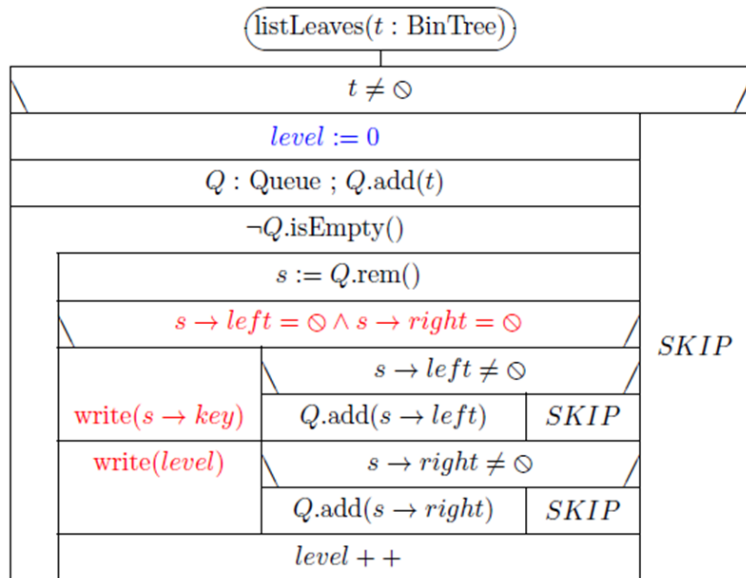
Melyik az a legkisebb szintszám ahol a fának levele van?

- **Bevezető feladat:** hogyan tarthatjuk nyilván a csúcsok szintszámát bejárás közben. Oldjuk meg elsőként azt a feladatot, hogy kilistázzuk a fa leveleiben található kulcsokat, a csúcs szintszámával együtt.

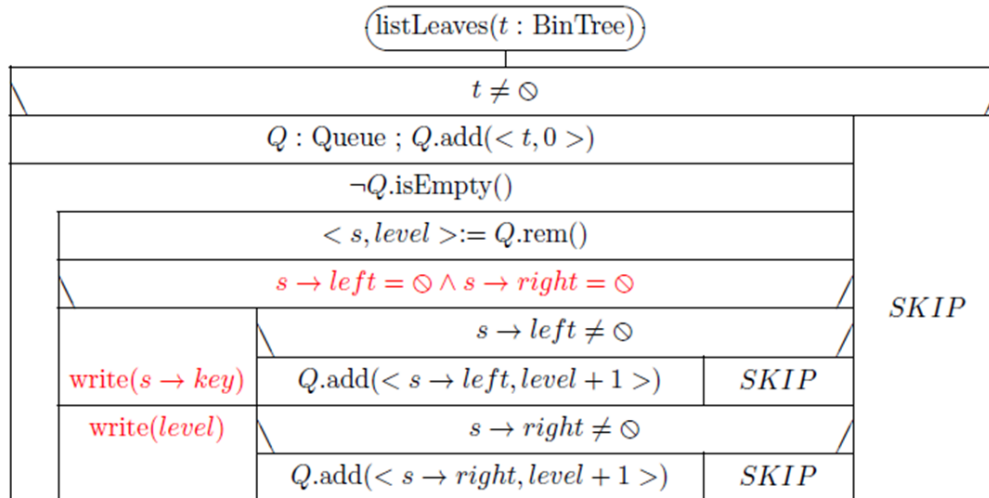
- Kézenfekvő, de rossz megoldás a következő:

Mi a hiba?

- Szint helyett azt írná ki, hogy mi a sorszáma a feldolgozott csúcsnak a szintfolytonos bejárás szerint.



- A sorba $\langle \text{Node}^*, N \rangle$ párokat teszünk, minden csúcs mellé betesszük a szintszámát is.

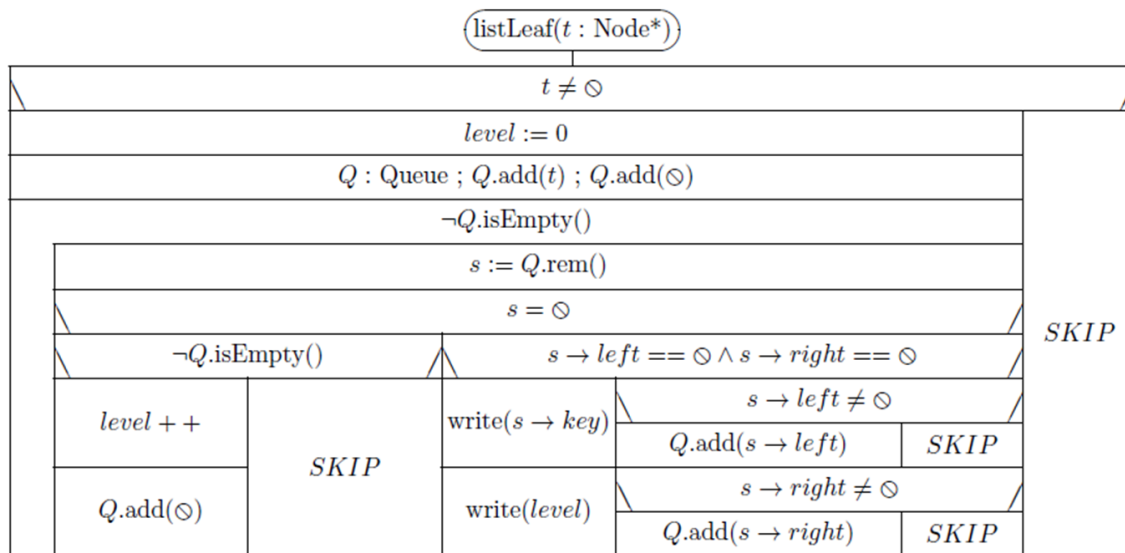


- **Kérdés:** megoldható anélkül is, hogy a szintszámot minden csúcsnál eltároljuk?

Igen, megoldható, két módszer is kínálkozik:

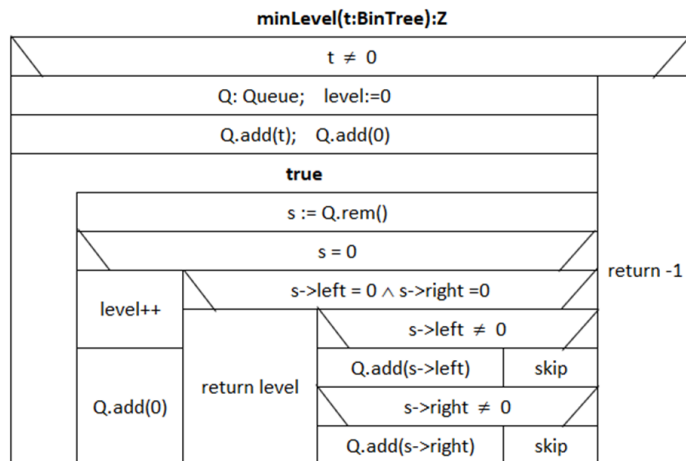
- Első:
 - Végjelekkel: minden szint végén egy végjelet (NULL) teszünk a sorba. Ezt úgy oldjuk meg, hogy kezdetben a gyökérelem után teszünk egy végjelet, majd amikor végjelet veszünk ki, megnöveljük a szintet, és beteszünk egy új végjelet a sorba.
- Második:
 - Szintek elemszámainak számolásával: a végjelek helyett számoljuk az elemeket. Mindig nyilvántartjuk azt, hogy
 - a) hány elem van ezen a szinten (ezelőtt kiszámoltuk),
 - b) a szinten belül hányadik elemnél tartunk,
 - c) számláljuk a következő szint elemszámát (ahány elemet beteszünk a sorba ezen a szinten).

Első (végjeles módszer) a levelek listázára



Végjeles módszerrel az eredeti feladat megoldása

„Mi az a legkisebb szint, ahol van levél?”

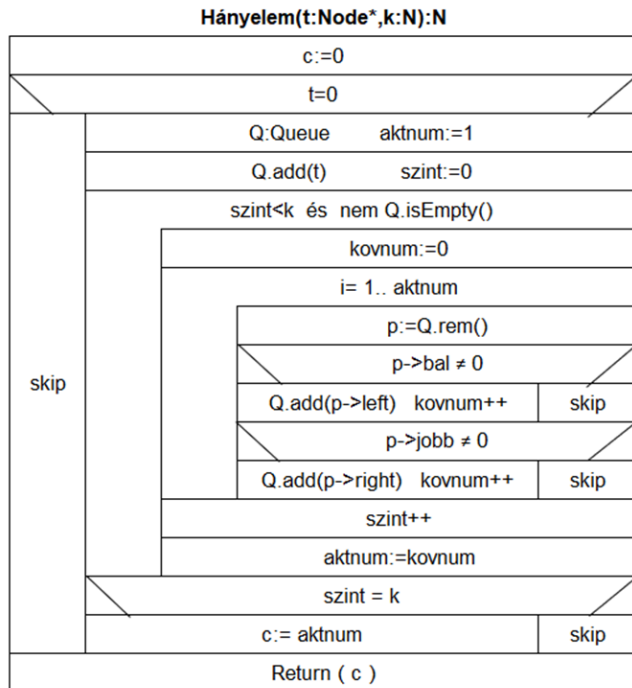


Miért nem végtelen ciklus?

A feladat valójában egy kiválasztás, mivel ha nem üres a fa, biztosan van levele, tehát a szintfolytonos bejárás garantáltan elérkezik egy levélhez, és mivel szintfolytonosan haladunk, ez a levél lesz az egyik a gyökérhez legközelebb esők közül.

A számlálós módszer bemutatása egy másik feladaton keresztül

- **Feladat:** hány csúcsa van a bináris fának a k . szinten?



aktnum - az aktuális szint csúcsainak száma

szint - az aktuális szint szintszáma

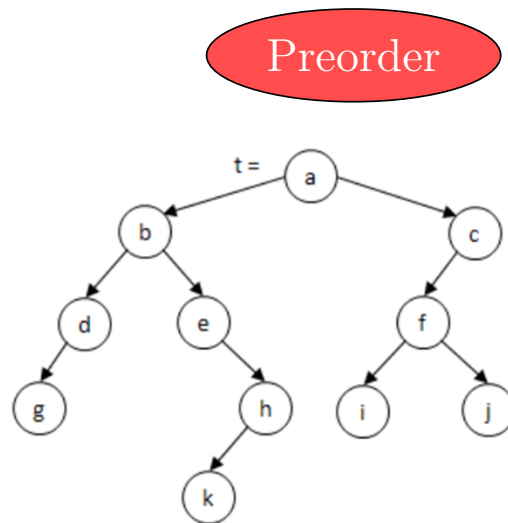
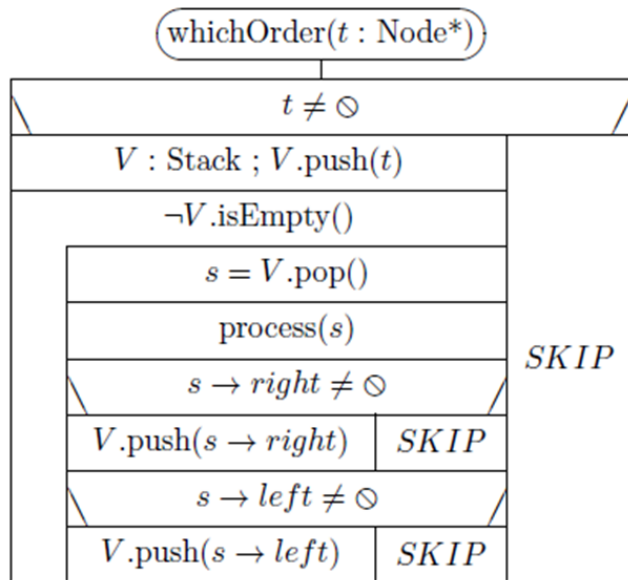
kovnum - miközben feldolgozzuk az aktuális szint elemeit, kovnum-ban megszámoljuk, hány csúcsot raktunk be a sorba, ennyi csúcs van a következő szinten.

Ha egy szinttel végeztünk, megnöveljük a szint-et, aktnum-ot frissítjük.

Ha elértük a k -dik szintet, akkor aktnum adja meg a csúcsok számát, ha nem értük el, akkor nincs k -dik szint, nulla lesz az eredmény.

Preorder vermekkel

- Módosítsuk úgy a szintfolytonos bejárást, hogy a sor helyett vermet használunk, és a gyerekek feldolgozási sorrendjét felcseréljük, előbb a jobb-, aztán a bal gyereket tesszük a verembe (ha van). Ekkor az egyik rekurzív bejáró algoritmus iteratív változatát kapjuk.
- Melyiket?



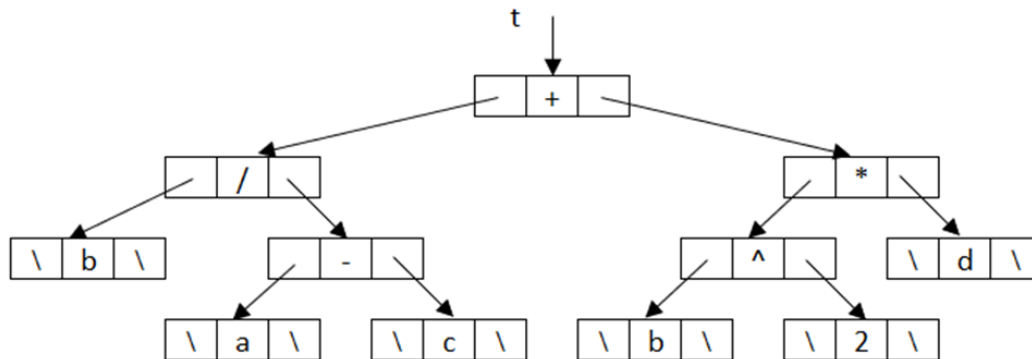
- Adott egy láncoltan ábrázolt bináris fa. Készítsen rekurzív algoritmust (nem szintfolytonos bejárással), mely megadja azt a legkisebb szintszámot, ahol a fának levele van. Üres fára -1-et kell visszaadnia.
- Adott egy láncoltan ábrázolt kifejezés fa. Készítsen rekurzív algoritmust, mely a kifejezés fát bejárva, kiírja a kifejezés teljesen zárójelezett alakját.

Kifejezés fa

Kifejezés fa (Expression Tree)

- Feltéve, hogy egy aritmetikai kifejezésben két-, esetleg egyoperandusú műveleti jelek (operátorok) fordulhatnak elő, a kifejezés ábrázolható egy bináris fával, ezt hívjuk kifejezés fának.

A $b/(a-c)+b^2*d$ kifejezést ábrázoló kifejezés fa.



A kifejezés fa postorder bejárása épp a kifejezés lengyel formáját adja:

$b \ a \ c \ - \ / \ b \ 2 \ ^ \ d \ * \ +$

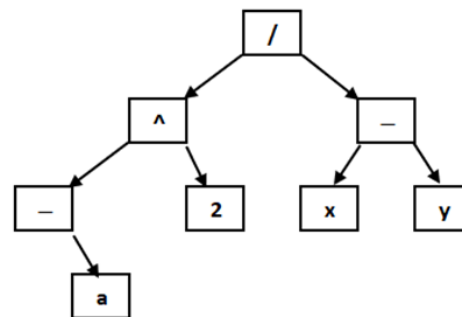
- A kifejezés fa összefügg a tanult lengyel formával, ugyanis megfigyelhető, hogy a kifejezés fa **postorder** bejárásával előállítható a lengyel forma, és fordítva is igaz, a lengyel formából felépíthető a kifejezés fa.

Kifejezésfa építése: Teljesen zárójelezett alakból

- **Teljesen (és helyesen) zárójelezett alak:** az eredeti helyesen zárójelezett kifejezésnek egy olyan alakja, melyben minden zárójelpár pontosan egy operátort tartalmaz, valamint a hozzá tartozó bal és jobb operandust. Az egymásba ágyazott zárójelpárok határozzák meg a műveletek elvégzésének sorrendjét.

- Példák:

- $a+b$ $(a+b)$
- $a+b*c$ $(a+(b*c))$
- $(a+b)/c$ $((a+b)/c)$
- $-a^2/(x-y)$ $(((-a)^2)/(x-y))$



- A **teljesen zárójelezett** kifejezésből könnyen felépíthetjük a **kifejezés fát**. Meg kell keresni a legkülső zárójelpárban található műveleti jelet, ez lesz a kifejezés fa gyökere. Bal részfáját az operátor bal oldali operandusából a jobb részfáját pedig a jobb operandusból építjük tovább ezen módon. Amikor az operandus már nem zárójelezett kifejezés, akkor belőle egy levél lesz.

Kifejezésfa felépítése a lengyel formából

- Legyen X egy lengyel formájú sorozat, elemei (x_i) vagy az operátorok, vagy az operandusok halmazába tartozik. Készítsük el a kifejezés fáját a lengyel formából.

lengyelbőlKiffa(X): Node*

V: Stack	
i = 1 to X.length()	
p := new Node (x _i)	
x _i ∈ {operátorok}	x _i ∈ {operandusok}
p->right:=V.pop()	skip
p->left:=V.pop()	
V.push(p)	
return V.pop()	

A verem a csúcsokra mutató pointereket

fogja tartalmazni.

A konstruktor elhelyezi a kulcsot az új Node-ban,

a left és right pointereket nullára állítja.

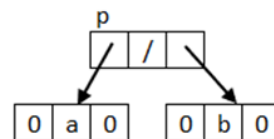
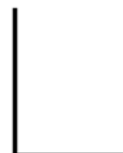
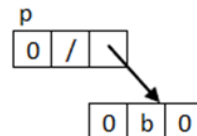
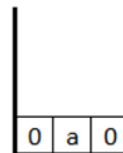
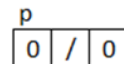
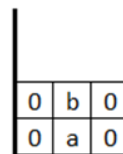
Operátor esetén már a veremben van a jobb,

alatta a bal operandust ábrázoló Node címe.

Berakjuk a verembe az új Node címét.

Végül a verem a fa gyökerére mutató pointert tartalmazza.

a b / feldolgozása



Valójában a veremben nem Node van, hanem a Node-ra mutató pointer

Köszönöm a figyelmet!